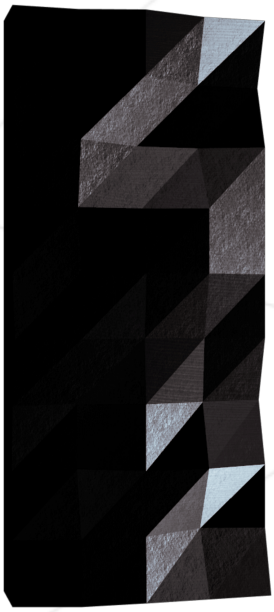

# Libft

## Your first own library

Pedago pedago@42.fr

*Summary: The aim of this project is to code a* `C` *library regrouping usual functions that you'll be allowed to use in all your other projects.*

# Contents


**I Foreword** **2**

**II Introduction** **3**

**III Objectives** **4**

**IV General Instructions** **5**

**V Mandatory part** **7**

V.1 Technical considerations . . . . . 7

V.2 Part 1 - Libc functions . . . . . 8

V.3 Part 2 - Additional functions . . . . . 10

**VI Bonus part** **15**

**VII Submission and peer correction** **18**

# Chapter I

# Foreword

This first project marks the beginning of your training to become de software engineer.

To accompany you during this project, here is a list of oustanding music groups. It's highly probable that you won't like any of those. This will mean that you have poor music taste. I'm sure that you have some other qualities such as being able to hold your breath for more than 3 minutes or maybe you know by heart the names of the 206 United Nations' signatory states. The groups aren't listed in any particular order and the list does not need to be exhaustive. Click on the links to find out more.

- Linkin Park
- Atreyu
- Infected Mushroom
- Nightwish
- Nightwish is great even if they change main singer every now and then
- Caliban
- Papa Roach
- Koan
- Staind
- Dream Theater
- Sphrongle

# Chapter II

# Introduction

The `libft` project builds on the concepts you learned during Day-06 of the bootcamp ie code a library of useful functions that you will be allowed to reuse in most of your `C` projects this year. This will save you a lot of precious time. The following assignments will have you write lines of code you already wrote during the bootcamp. See the `libft` project as a Bootcamp reminder and use it wisely to assess your level and progress.

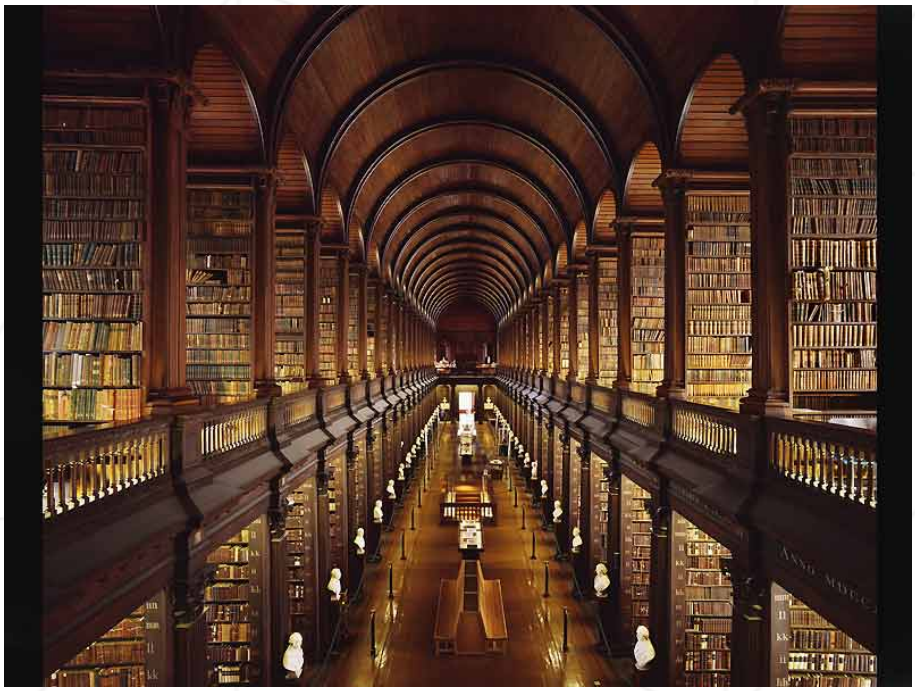

Figure II.1: A possible representation of your Libft (artist's view)

# Chapter III

# Objectives

`C` programming can be very tedious when one doesn't have access to those highly useful standard functions. This project makes you to take the time to re-write those functions, understand them, and learn to use them. This library will help you for all your future `C` projects.

Through this project, we also give you the opportunity to expand the list of functions with your own. Take the time to expand your `libft` throughout the year.

# Chapter IV

# General Instructions

- You must create the following functions in the order you believe makes most sense. We encourage you to use the functions you have already coded to write the next ones. The difficulty level does not increase by assignment and the project has not been structured in any specific way. It is similar to a video game, where you can complete quests in the order of your choosing and use the loot from the previous quests to solve the next ones.

- Your project must be written in accordance with the Norm.

- Your functions should not quit unexpectedly (segmentation fault, bus error, double free, etc) apart from undefined behaviors. If this happens, your project will be considered non functional and will receive a `0` during the defence.

- All heap allocated memory space must be properly freed when necessary.

- You must submit a file named `author` containing your username followed by a '\n' at the root of your repository,

```
$>cat -e author
xlogin$
```

- You must submit a `C` file for each function you create, as well as a `libft.h` file, which will contain all the necessary prototypes as well as `macros` and `typedefs` you might need. All those files must be at the root of your repository.

- You must submit a `Makefile` which will compile your source files to a static library `libft.a`.

- Your `Makefile` must at least contain the rules `$(NAME)`, `all`, `clean`, `fclean` et `re` in the order that you will see fit.

- Your `Makefile` must compile your work with the flags `-Wall`, `-Wextra` and `-Werror`.

- Only the following `libc` functions are allowed : `malloc(3)`, `free(3)` and `write(2)`, and their usage is restricted. See below.

- You must include the necessary `include` system files to use one or more of the three authorized functions in your `.c` files. The only additional system `include` file you are allowed to use is `string.h` to have access to the constant `NULL` and to the type `size_t`. Everything else if forbidden.

- We encourage you to create test programs for your library even though this work **won't have to be submitted and won't be graded**. It will give you a chance to easily test your work and your peers' work. You will find those tests especially useful during your defence. Indeed, during defence, you are free to use your tests and/or the tests of the peer you are correcting.

# Chapter V

# Mandatory part

## V.1 Technical considerations

- Your `libft.h` file can contain `macros` and `typedefs` if needed.
- A string must **ALWAYS** end with a `'\0'`, even if it is not included in the function's description, unless explicitly stated otherwise.
- It is forbidden to use global variables.
- If you need sub-functions to write a complex function, you must define these sub-functions as `static` as stipulated in the Norm.

> Check out this link to find out more about static functions:
> http://codingfreak.blogspot.com/2010/06/static-functions-in-c.html

- You must pay attention to your types and wisely use the casts when needed, especially when a `void*` type is involved. Generally speaking, avoid implicit casts. Example:

```
char    *str;

str = malloc(42 * sizeof(*str));            /* Wrong ! Malloc returns a void * (implicit cast) */
str = (char *) malloc(42 * sizeof(*str));   /* Right ! (explicit cast) */
```

## V.2 Part 1 - Libc functions

In this first part, you must re-code a set of the `libc` functions, as defined in their `man`. Your functions will need to present the same prototype and behaviors as the originals. Your functions' names must be prefixed by "`ft_`". For instance `strlen` becomes `ft_strlen`.

> Some of the functions' prototypes you have to re-code use the "restrict" qualifier. This keyword is part of the c99 standard. It is therefore forbidden to include it in your prototypes and to compile it with the flag -std=c99.

You must re-code the following functions:

- `memset`
- `bzero`
- `memcpy`
- `memccpy`
- `memmove`
- `memchr`
- `memcmp`
- `strlen`
- `strdup`
- `strcpy`
- `strncpy`
- `strcat`
- `strncat`
- `strlcat`
- `strchr`
- `strrchr`
- `strstr`
- `strnstr`
- `strcmp`
- `strncmp`
- `atoi`
- `isalpha`

- `isdigit`
- `isalnum`
- `isascii`
- `isprint`
- `toupper`
- `tolower`

## V.3 Part 2 - Additional functions

In this second part, you must code a set of functions that are either not included in the `libc`, or included in a different form. Some of these functions can be useful to write Part 1's functions.

- | **ft_memalloc** | |
|---|---|
| **Prototype** | `void * ft_memalloc(size_t size);` |
| **Description** | Allocates (with `malloc(3)`) and returns a "fresh" memory area. The memory allocated is initialized to `0`. If the allocation fails, the function returns `NULL`. |
| **Param. #1** | The size of the memory that needs to be allocated. |
| **Return value** | The allocated memory area. |
| **Libc functions** | `malloc(3)` |

- | **ft_memdel** | |
|---|---|
| **Prototype** | `void ft_memdel(void **ap);` |
| **Description** | Takes as a parameter the address of a memory area that needs to be freed with `free(3)`, then puts the pointer to `NULL`. |
| **Param. #1** | A pointer's address that needs its memory freed and set to `NULL`. |
| **Return value** | None. |
| **Libc functions** | `free(3)`. |

- | **ft_strnew** | |
|---|---|
| **Prototype** | `char * ft_strnew(size_t size);` |
| **Description** | Allocates (with `malloc(3)`) and returns a "fresh" string ending with `'\0'`. Each character of the string is initialized at `'\0'`. If the allocation fails the function returns `NULL`. |
| **Param. #1** | The size of the string to be allocated. |
| **Return value** | The string allocated and initialized to `0`. |
| **Libc functions** | `malloc(3)` |

- | **ft_strdel** | |
|---|---|
| **Prototype** | `void ft_strdel(char **as);` |
| **Description** | Takes as a parameter the address of a string that need to be freed with `free(3)`, then sets its pointer to `NULL`. |
| **Param. #1** | The string's address that needs to be freed and its pointer set to `NULL`. |
| **Return value** | None. |
| **Libc functions** | `Free(3)`. |

- | **ft_strclr** | |
|---|---|
| **Prototype** | `void ft_strclr(char *s);` |
| **Description** | Sets every character of the string to the value `'\0'`. |
| **Param. #1** | The string that needs to be cleared. |
| **Return value** | None. |
| **Libc functions** | None. |

- | ft_striter | |
|---|---|
| **Prototype** | `void ft_striter(char *s, void (*f)(char *));` |
| **Description** | Applies the function `f` to each character of the string passed as argument. Each character is passed by address to `f` to be modified if necessary. |
| **Param. #1** | The string to iterate. |
| **Param. #2** | The function to apply to each character of `s`. |
| **Return value** | None. |
| **Libc functions** | None. |

- | ft_striteri | |
|---|---|
| **Prototype** | `void ft_striteri(char *s, void (*f)(unsigned int, char *));` |
| **Description** | Applies the function `f` to each character of the string passed as argument, and passing its index as first argument. Each character is passed by address to `f` to be modified if necessary. |
| **Param. #1** | The string to iterate. |
| **Param. #2** | The function to apply to each character of `s` and its index. |
| **Return value** | None. |
| **Libc functions** | None. |

- | ft_strmap | |
|---|---|
| **Prototype** | `char * ft_strmap(char const *s, char (*f)(char));` |
| **Description** | Applies the function `f` to each character of the string given as argument to create a "fresh" new string (with `malloc(3)`) resulting from the successive applications of `f`. |
| **Param. #1** | The string to map. |
| **Param. #2** | The function to apply to each character of `s`. |
| **Return value** | The "fresh" string created from the successive applications of `f`. |
| **Libc functions** | `malloc(3)` |

- | ft_strmapi | |
|---|---|
| **Prototype** | `char * ft_strmapi(char const *s, char (*f)(unsigned int, char));` |
| **Description** | Applies the function `f` to each character of the string passed as argument by giving its index as first argument to create a "fresh" new string (with `malloc(3)`) resulting from the successive applications of `f`. |
| **Param. #1** | The string to map. |
| **Param. #2** | The function to apply to each character of `s` and its index. |
| **Return value** | The "fresh" string created from the successive applications of `f`. |
| **Libc functions** | `malloc(3)` |

- | ft_strequ | |
|---|---|
| **Prototype** | `int ft_strequ(char const *s1, char const *s2);` |
| **Description** | Lexicographical comparison between `s1` and `s2`. If the 2 strings are identical the function returns `1`, or `0` otherwise. |
| **Param. #1** | The first string to be compared. |
| **Param. #2** | The second string to be compared. |
| **Return value** | `1` or `0` according to if the 2 strings are identical or not. |
| **Libc functions** | None. |

- | ft_strnequ | |
|---|---|
| **Prototype** | `int ft_strnequ(char const *s1, char const *s2, size_t n);` |
| **Description** | Lexicographical comparison between `s1` and `s2` up to `n` characters or until a `'\0'` is reached. If the 2 strings are identical, the function returns `1`, or `0` otherwise. |
| **Param. #1** | The first string to be compared. |
| **Param. #2** | The second string to be compared. |
| **Param. #3** | The maximum number of characters to be compared. |
| **Return value** | `1` or `0` according to if the 2 strings are identical or not. |
| **Libc functions** | None. |

- | ft_strsub | |
|---|---|
| **Prototype** | `char * ft_strsub(char const *s, unsigned int start, size_t len);` |
| **Description** | Allocates (with `malloc(3)`) and returns a "fresh" substring from the string given as argument. The substring begins at index`start` and is of size `len`. If `start` and `len` aren't refering to a valid substring, the behavior is undefined. If the allocation fails, the function returns `NULL`. |
| **Param. #1** | The string from which create the substring. |
| **Param. #2** | The start index of the substring. |
| **Param. #3** | The size of the substring. |
| **Return value** | The substring. |
| **Libc functions** | `malloc(3)` |

- | ft_strjoin | |
|---|---|
| **Prototype** | `char * ft_strjoin(char const *s1, char const *s2);` |
| **Description** | Allocates (with `malloc(3)`) and returns a "fresh" string ending with `'\0'`, result of the concatenation of `s1` and `s2`. If the allocation fails the function returns `NULL`. |
| **Param. #1** | The prefix string. |
| **Param. #2** | The suffix string. |
| **Return value** | The "fresh" string result of the concatenation of the 2 strings. |
| **Libc functions** | `malloc(3)` |

- | ft_strtrim | |
|---|---|
| **Prototype** | `char * ft_strtrim(char const *s);` |
| **Description** | Allocates (with `malloc(3)`) and returns a copy of the string given as argument without whitespaces at the beginning or at the end of the string. Will be considered as whitespaces the following characters `' '`, `'\n'` and `'\t'`. If `s` has no whitespaces at the beginning or at the end, the function returns a copy of `s`. If the allocation fails the function returns `NULL`. |
| **Param. #1** | The string to be trimed. |
| **Return value** | The "fresh" trimmed string or a copy of `s`. |
| **Libc functions** | `malloc(3)` |

- | ft_strsplit | |
|---|---|
| **Prototype** | `char ** ft_strsplit(char const *s, char c);` |
| **Description** | Allocates (with `malloc(3)`) and returns an array of "fresh" strings (all ending with `'\0'`, including the array itself) obtained by spliting `s` using the character `c` as a delimiter. If the allocation fails the function returns `NULL`. Example : `ft_strsplit("*hello*fellow***students*", '*')` returns the array `["hello", "fellow", "students"]`. |
| **Param. #1** | The string to split. |
| **Param. #2** | The delimiter character. |
| **Return value** | The array of "fresh" strings result of the split. |
| **Libc functions** | `malloc(3)`, `free(3)` |

- | ft_itoa | |
|---|---|
| **Prototype** | `char * ft_itoa(int n);` |
| **Description** | Allocate (with `malloc(3)`) and returns a "fresh" string ending with `'\0'` representing the integer `n` given as argument. Negative numbers must be supported. If the allocation fails, the function returns `NULL`. |
| **Param. #1** | The integer to be transformed into a string. |
| **Return value** | The string representing the integer passed as argument. |
| **Libc functions** | `malloc(3)` |

- | ft_putchar | |
|---|---|
| **Prototype** | `void ft_putchar(char c);` |
| **Description** | Outputs the character `c` to the standard output. |
| **Param. #1** | The character to output. |
| **Return value** | None. |
| **Libc functions** | `write(2)`. |

- | ft_putstr | |
|---|---|
| **Prototype** | `void ft_putstr(char const *s);` |
| **Description** | Outputs the string `s` to the standard output. |
| **Param. #1** | The string to output. |
| **Return value** | None. |
| **Libc functions** | `write(2)`. |

- | ft_putendl | |
|---|---|
| **Prototype** | `void ft_putendl(char const *s);` |
| **Description** | Outputs the string `s` to the standard output followed by a `'\n'`. |
| **Param. #1** | The string to output. |
| **Return value** | None. |
| **Libc functions** | `write(2)`. |

- | ft_putnbr | |
|---|---|
| **Prototype** | `void ft_putnbr(int n);` |
| **Description** | Outputs the integer `n` to the standard output. |
| **Param. #1** | The integer to output. |
| **Return value** | None. |
| **Libc functions** | `write(2)`. |

- | ft_putchar_fd | |
|---|---|
| **Prototype** | `void ft_putchar_fd(char c, int fd);` |
| **Description** | Outputs the char `c` to the file descriptor `fd`. |
| **Param. #1** | The character to output. |
| **Param. #2** | The file descriptor. |
| **Return value** | None. |
| **Libc functions** | `write(2)`. |

- | ft_putstr_fd | |
|---|---|
| **Prototype** | `void ft_putstr_fd(char const *s, int fd);` |
| **Description** | Outputs the string `s` to the file descriptor `fd`. |
| **Param. #1** | The string to output. |
| **Param. #2** | The file descriptor. |
| **Return value** | None. |
| **Libc functions** | `write(2)`. |

- | ft_putendl_fd | |
|---|---|
| **Prototype** | `void ft_putendl_fd(char const *s, int fd);` |
| **Description** | Outputs the string `s` to the file descriptor `fd` followed by a `'\n'`. |
| **Param. #1** | The string to output. |
| **Param. #2** | The file descriptor. |
| **Return value** | None. |
| **Libc functions** | `write(2)`. |

- | ft_putnbr_fd | |
|---|---|
| **Prototype** | `void ft_putnbr_fd(int n, int fd);` |
| **Description** | Outputs the integer `n` to the file descriptor `fd`. |
| **Param. #1** | The integer to print. |
| **Param. #2** | The file descriptor. |
| **Return value** | None. |
| **Libc functions** | `write(2)`. |

# Chapter VI

# Bonus part

If you successfully completed the mandatory part, you'll enjoy taking it further. You can see this last section as Bonus Points.

Having functions to manipulate memory and strings is very useful, but you'll soon discover that having functions to manipulate lists is even more useful.

You'll use the following structure to represent the links of your list. This structure must be added to your `libft.h` file.

```
typedef struct      s_list
{
    void            *content;
    size_t          content_size;
    struct s_list   *next;
}                   t_list;
```

Here is a description of the fields of the `t_list` struct:

- `content` : The data contained in the link. The `void *` allows to store any kind of data.
- `content_size` : The size of the data stored. The `void *` type doesn't allow you to know the size of the pointed data, as a consequence, it is necessary to save its size. For instance, the size of the string `"42"` is `3 bytes` and the 32bits integer `42` has a size of `4 bytes`.
- `next` : The next link's address or `NULL` if it's the last link.

The following functions will allow you to manipulate your lists more easilly.

- | ft_lstnew | |
|---|---|
| **Prototype** | `t_list * ft_lstnew(void const *content, size_t content_size);` |
| **Description** | Allocates (with `malloc(3)`) and returns a "fresh" link. The variables `content` and `content_size` of the new link are initialized by **copy** of the parameters of the function. If the parameter `content` is nul, the variable `content` is initialized to `NULL` and the variable `content_size` is initialized to `0` even if the parameter `content_size` isn't. The variable `next` is initialized to `NULL`. If the allocation fails, the function returns `NULL`. |
| **Param. #1** | The content to put in the new link. |
| **Param. #2** | The size of the content of the new link. |
| **Return value** | The new link. |
| **Libc functions** | `malloc(3)`, `free(3)` |

- | ft_lstdelone | |
|---|---|
| **Prototype** | `void ft_lstdelone(t_list **alst, void (*del)(void *, size_t));` |
| **Description** | Takes as a parameter a link's pointer address and frees the memory of the link's content using the function `del` given as a parameter, then frees the link's memory using `free(3)`. The memory of `next` musnt not be freed under any circumstance. Finally, the pointer to the link that was just freed must be set to `NULL` (quite similar to the function `ft_memdel` in the mandatory part). |
| **Param. #1** | The adress of a pointer to a link that needs to be freed. |
| **Return value** | None. |
| **Libc functions** | `free(3)` |

- | ft_lstdel | |
|---|---|
| **Prototype** | `void ft_lstdel(t_list **alst, void (*del)(void *, size_t));` |
| **Description** | Takes as a parameter the adress of a pointer to a link and frees the memory of this link and every successors of that link using the functions `del` and `free(3)`. Finally the pointer to the link that was just freed must be set to `NULL` (quite similar to the function `ft_memdel` from the mandatory part). |
| **Param. #1** | The address of a pointer to the first link of a list that needs to be freed. |
| **Return value** | None. |
| **Libc functions** | `free(3)` |

- | ft_lstadd | |
|---|---|
| **Prototype** | `void ft_lstadd(t_list **alst, t_list *new);` |
| **Description** | Adds the element `new` at the beginnning of the list. |
| **Param. #1** | The address of a pointer to the first link of a list. |
| **Param. #2** | The link to add at the beginning of the list. |
| **Return value** | None. |
| **Libc functions** | None. |

- | ft_lstiter | |
|---|---|
| **Prototype** | `void ft_lstiter(t_list *lst, void (*f)(t_list *elem));` |
| **Description** | Iterates the list `lst` and applies the function `f` to each link. |
| **Param. #1** | A pointer to the first link of a list. |
| **Param. #2** | The address of a function to apply to each link of a list. |
| **Return value** | None. |
| **Libc functions** | None. |

- | ft_lstmap | |
|---|---|
| **Prototype** | `t_list * ft_lstmap(t_list *lst, t_list * (*f)(t_list *elem));` |
| **Description** | Iterates a list `lst` and applies the function `f` to each link to create a "fresh" list (using `malloc(3)`) resulting from the successive applications of `f`. If the allocation fails, the function returns `NULL`. |
| **Param. #1** | A pointer's to the first link of a list. |
| **Param. #2** | The address of a function to apply to each link of a list. |
| **Return value** | The new list. |
| **Libc functions** | `malloc(3)`, `free(3)`. |

If you successfully completed both the mandatory and bonus sections of this project, we encourage you to add other functions that you believe could be useful to expand your library. For instance, a version of `ft_strsplit` that returns a list instead of an array, the function `ft_lstfold` similar to the function `reduce` in `Python` and the function `List.fold_left` in `OCaml` (beware of the memory leak !). You can add functions to manipulate arrays, stacks, files, maps, hashtables, etc. The limit is your imagination.

# Chapter VII

# Submission and peer correction

Submit your work on your `GiT` repository as usual. Only the work on your repository will be graded.

Once your have completed your defences, Deepthought (the "moulinette") will grade your work. Your final grade will be calculated taking into account your peer-correction grades and Deepthought's grade.

Deepthought will grade your assignments in the order of the subject : Part 1, Part 2 and Bonus. One error in one of the sections will automatically stop the grading.

Good luck to you and don't forget your author file !